

Foundations of Programming Languages

Exercises 1

Arithmetic Expressions

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

The exercise numbers in this list refer to the exercise numbers in book *Types and Programming Languages*✶. Benjamin Pierce, The MIT Press, 2002.

Pencil and paper

1. Given the language of terms in page 24, write terms for

- a few natural numbers $(1, 2, 3, \dots)$,
- expressions with two or more conditionals,
- expressions that mix arithmetic and boolean operators,
- `not t`, `t1 or t2`, `t1 and t2`,
- `isone t`, `gz t` (that is, greater-than-zero)

The expressions in last two bullets should be encoded in the language of terms (that in page 24).

2. Using the rules in Definition 3.2.2 show that
`if (iszero(succ 0)) then false else (succ false) ∈  $\mathcal{T}$ .`
3. Show that
`succ(if (iszero succ) then false else (iszero false)) ∉  $\mathcal{T}$ .`
4. Write arithmetic expressions that evaluate and that do not evaluate according to the rules in Figure 3-1.
5. The *width* of a term is a) 1 for each constant, b) the width of t for terms `succ t`, `pred t` and `iszero t`, and c) the sum of the widths of the t_1 , t_2 and t_3 for term `if  $t_1$  then  $t_2$  else  $t_3$` . Present an inductive definition of *width*(t) in the style of Definitions 3.3.1 and 3.3.2.

Programming The following exercises can be based on the Haskell source code adapted from TAPL and provided at the course's web site.

6. Write Haskell terms for the arithmetic expressions in item 1.
7. Write Haskell terms for the expressions in item 4. Take one of the expressions that does not evaluate and call it t . What happens if we try evaluating the Haskell expression `step t`?
8. Write a function for `size`, Definition 3.3.2.
9. Write a function for `depth`, Definition 3.3.2.
10. Write a function for `width`, item 5.